

实验二：UNIX V6++ 内核的启动——从 BIOS 到 Bootloader

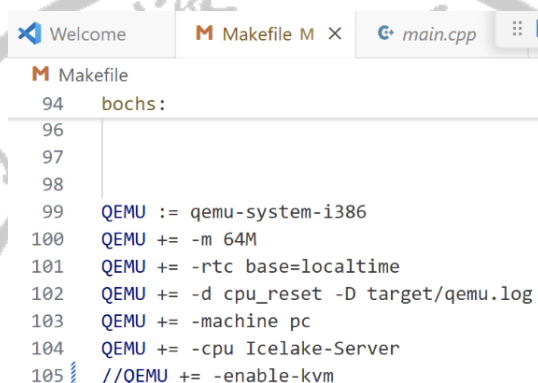
1. 实验目的

本次实验将指导读者使用 QEMU 模拟器来研究 IA-32 兼容计算机的启动方式。在这一过程中，读者将熟悉系统如何从上电到成功加载内核，如何从实模式进入到保护模式。

2. 实验准备

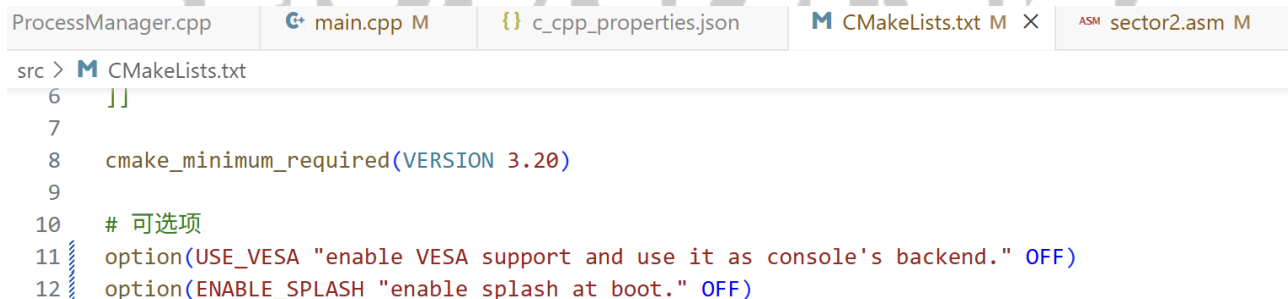
本次实验开始之前，需要完成如下的准备工作：

(1) 确认 Makefile 文件中，下图所示的最后一条语句已经被注释掉。



```
94 bochs:
95
96
97
98
99 QEMU := qemu-system-i386
100 QEMU += -m 64M
101 QEMU += -rtc base=localtime
102 QEMU += -d cpu_reset -D target/qemu.log
103 QEMU += -machine pc
104 QEMU += -cpu Icelake-Server
105 //QEMU += -enable-kvm
```

(2) 找到 CMakeLists.txt 文件，将下图中的两个可选项由“ON”改为“OFF”。



```
src > CMakeLists.txt
6
7
8 cmake_minimum_required(VERSION 3.20)
9
10 # 可选项
11 option(USE_VESA "enable VESA support and use it as console's backend." OFF)
12 option(ENABLE_SPLASH "enable splash at boot." OFF)
```

(3) 将 src/boot 文件夹下的 boot.asm 和 sector2.asm 两个文件的原有内容分别复制出来做好备份，以防此次实验不成功，你的内核将无法再启动。

(4) 用下面的代码代替 sector2.asm 文件中原有的代码，并将 boot.asm 原有的代码全部删除。

代码 1: sector2.asm

```
[BITS 32]
[extern kernelBridge]
global greatstart
greatstart:
jmp kernelBridge
```

3. 预备知识

3.1. 从保护模式到实模式

3.1.1. 实模式

1978 年的 8086 芯片是 x86 架构的首款芯片。8086 CPU 中所有的寄存器的结构为 16 位，即每个寄存器中可以存放下 2 个字节，称 8086 的工作模式是 16 位模式，这就是实模式（Real Mode）。8086 的后面 CPU 为保持兼容，在芯片刚上电后，须首先运行在 16 位模式下。

寄存器是 CPU 内部用于存储信息的单元，是 CPU 中的稀有资源，对于一个汇编程序员来说，CPU 中主要可以使用的也就是寄存器。汇编程序员可以使用指令来读写 CPU 中的寄存器，从而可以实现对于 CPU 的控制。8086 CPU 中，寄存器的个数有 14 个，这 14 个寄存器是：AX, BX, CX, DX, SP, BP, SI, DI, IP, PSW, CS, DS, SS, ES，而这 14 个寄存器按照一定方式又分为了通用寄存器，控制寄存器和段寄存器。

AX、BX、CX、DX、SI、DI、SP、BP 这 8 个寄存器通常用来存放一般性的数据，被称为**通用寄存器**：

- AX (Accumulator)：累加器，用于算术运算和 I/O 操作。可分为两个 8 位的 AH 和 AL。
- BX (Base)：基址寄存器，用于内存寻址。可分为两个 8 位的 BH 和 BL。
- CX (Count)：计数器，用于循环或重复操作的次数控制。可分为两个 8 位的 CH 和 CL。
- DX (Data)：数据寄存器，用于 I/O 端口操作或与大数据运算配合使用。可分为两个 8 位的 DH 和 DL。
- SP (Stack Pointer)：堆栈指针，指向当前堆栈顶部地址。
- BP (Base Pointer)：基址指针，用于访问堆栈中的参数或局部变量。
- SI (Source Index)：源变址寄存器，用于字符串/数组操作的源地址索引。
- DI (Destination Index)：目标变址寄存器，用于字符串/数组操作的目标地址索引。

需要注意的是：

(1) 尽管这 8 个通用寄存器大多时候可以用作任何用途，但是在某些情况下，它们也有隐含的用法。比如 CX 可默认用于循环计数器，在使用 LOOP 指令循环时用来指定循环次数的寄存器；DI 和 SI 常用于串操作，DI 用于寻址目标数据串，SI 用于寻址源数据串；BP 和 SP 主要用来维护栈，SP 通常指向栈的顶部，BP 指向当前栈帧的起始地址。

(2) AX 可作为累加器用于乘法、除法及一些调整指令，也可以保存被访问存储器单元的偏移地址。BX 常用于地址指针，保存被访问存储器单元的偏移地址。CX 经常用作计数器，用于保存指令的计数值，也可以保存访问数据所在存储器单元的偏移地址。DX 常与 EAX 配合，用于保存乘法形成的部分结果，或者除法操作前的被除数，它还可以保存寻址存储器数据。

还有一组寄存器用来存放段基址，被称为**段寄存器**：

- CS (Code Segment)：代码段寄存器，存储指令的段地址。
- DS (Data Segment)：数据段寄存器，存储要访问数据的段地址。
- SS (Stack Segment)：栈段寄存器，存储栈的段地址。
- ES (Extra Segment)：附加段寄存器

最后两个寄存器是专门用来配合 CPU 完成指令寻址和记录 CPU 执行状态的：

- IP (Instruction Pointer)：指令指针寄存器，用于指示下一条执行指令的地址。
- PSW：标志寄存器，用于存放运算结果的状态标志。

实模式下，在内存管理上，CPU 使用程序地址直接访问物理内存，然而，8086 的地址总线是 20 位的，也就是内存的寻址空间有 $2^{20} = 1\text{M}$ 。这 1M 的地址空间分布详见表 1。这 1M 空间中的高 384KB 区域（从 0x000A0000 到 0x000FFFFF）由硬件保留，用于特殊用途，如视频显示缓冲区和非易失性存储器中的固件。这个保留区域最重要的部分是基本输入/输出系统(BIOS)，它占据了从 0x000F0000 到 0x000FFFFF 的 64KB 区域。BIOS 负责执行基本的系统初始化。

表 1：PC 的 1M 物理内存空间分布

起始地址	大小	用途
0x000000	1KB	BIOS 的中断向量表
0x000400	256B	BIOS 数据区
0x000500	29KB+768B	可用区域
0x007C00	512B	引导扇区数据，被 BIOS 加载到此处
0x007E00	607KB+512B	可用区域
0x009FC00	1KB	扩展 BIOS 数据区
0x00A0000	64KB	彩色显示适配器缓存
0x00B0000	32KB	黑白显示适配器缓存
0x00B8000	32KB	文本模式显示适配器缓存
0x00C0000	32KB	显示适配器 BIOS
0x00C8000	160KB	映射硬件适配器 ROM 或内存映射式的 IO
0x00F0000	64KB	BIOS 程序，入口地址在 0xFFFF0

这样问题就来了，8086 的寄存器是 16 位的，导致程序员在写程序时，无法把一个 20 位的地址完整放到一个寄存器里。为解决这问题，8086 引入了段寄存器，如 CS、DS、ES、GS、SS。段寄存器记录了一个段基地址，程序地址用“段寄存器:段内偏移”表示，通过计算可得到存储的真实地址（物理地址），计算的过程如图 1 所示。

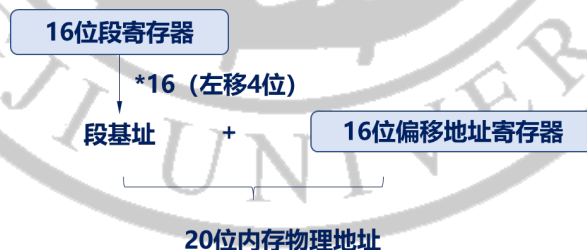


图 1：16 位实模式下程序逻辑地址到 20 位内存物理地址的计算过程

8086 中，CS 被用来作为代码段基址寄存器，DS 被用来作为数据段基址寄存器。也就是说，寻址时，无需显式声明数据段基址存储在段寄存器 DS，但 CPU 在执行时会默认使用 DS 做数据段寄存器；类似的 SS，作为栈基址寄存器，使用 PUSH 指令时，要保存的数据会放在 SS:SP。这按功能分段的管理内存方式就是段式管理。

当 Intel 最终凭借分别支持 16MB 和 4GB 物理地址空间的 80286 和 80386 处理器“打破了 1M 字节的障碍”时，PC 架构师却选择保留了低 1MB 物理地址空间的原始布局，以确保与现有软件的向后兼容性。

3.1.2. 保护模式

X86 CPU 历史上使用最广泛、影响力最大的是 32 位 CPU-i386 芯片。它与 8086 的不同就是采用全新的保护模式。保护模式出现的主要原因是为了解决实模式的两个问题：

- (1) CPU 对任何指令不加区分地执行
- (2) CPU 对访问内存的地址不加限制

那接下来，让我们一起来看看保护模式相较于实模式都做了哪些改变。

首先，保护模式增加了部分寄存器位宽，同时新增了一些寄存器，如表 2 所示。

表 2：X86 保护模式寄存器

寄存器	描述
EAX, EBX, ECX, EDX, EDI, ESI, EBP	32 位通用寄存器，里面可以存放数据、地址、参与运算
EIP	32 位程序指针寄存器，始终指向下一条指令的地址
ESP	栈指针寄存器，始终指向当前栈顶
CS, DS, ES, SS, FS, GS	16 位段寄存器，里面存放一个内存段的描述符索引
EFLAGS	32 位 CPU 状态寄存器
CRO, CR1, CR2, CR3	32 位 CPU 控制寄存器

i386 的地址总线是 32 位，通用寄存器也变成 32 位，因为寄存器位数不够而产生的段基址寄存器已失去作用。但是 i386 没有放弃段寄存器，而将它进化成了新的段式内存管理。段寄存器仍然是 16 位寄存器，但是其中存的不再是段基址，而是被称为段选择子（Segment Selector）的东西。相比 8086 芯片，i386 中多了个全局描述符表（Global Descriptor Table, GDT）。本质是个数组，其中每项都是一个段描述符，32 位段基址就存储在这个描述符中。而段选择子就是这个数组的下标。

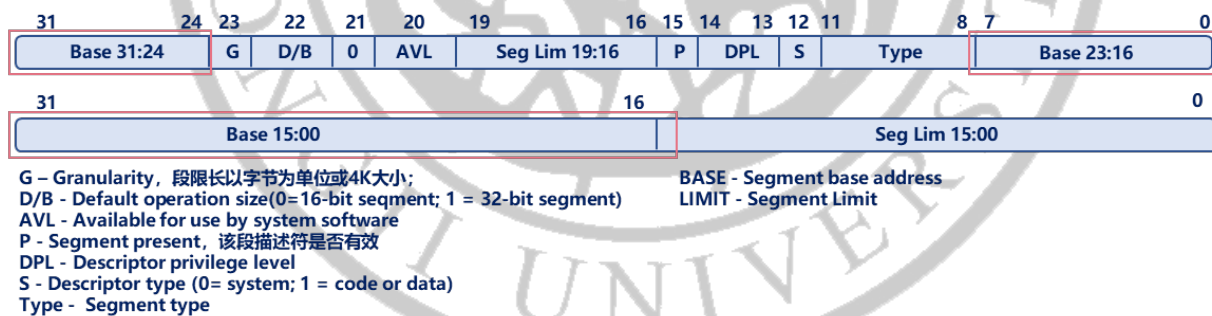


图 2：段描述符构成

GDT 在保护模式中的地位极高，没有 GDT 就没有强大的寻址能力，保护模式的优越性也荡然无存。GDT 中每个段描述符占 8 个字节，其结构和各标志位的含义如图 2 和表 3 所示。在段描述符中我们最关心的部分只有段基址与段界限。32 位的段基址实现了 4G 的超大寻址空间，而段界限限制了段的大小，保护了不同的物理地址段之间不能越界访问。这些不正是我们在之前标榜的保护模式的强大功能吗！

表 3：段描述符中各标志位含义

标志位	含义
G	用于表示段限长所用的单位 G=0, 段限长单位为字节；G=1, 段限长单位为页（4KB）

D/B	代码段：D=1，代码段默认的操作和寻址尺寸均为 32 位； D=0，代表代码段是 16 位的 80286 寻址模式，操作和寻址尺寸均为 16 位 堆栈段：B=1，隐含的堆栈操作为 32 位，堆栈指针用 ESP B=0，堆栈操作为 16 位，堆栈指针用 SP 数据段：B=1，表示数据位 32 位；B=0，表示数据位 16 位																																																																																																																																			
AVL	保留位，可以被系统软件使用 AVL=0，系统软件不可使用本段；AVL=1，系统软件可使用本段																																																																																																																																			
P	存在位 P=1，该段在内存中；P=0，该段不在内存																																																																																																																																			
DPL	描述符特权级位，占两位。这两位可表示 0~3 级特权，表示该段访问的特权级																																																																																																																																			
S	描述符类型位 S=1，数据/代码段描述符；S=0，系统段/门描述符																																																																																																																																			
Type	段类型位，其具体含义如下图所示 <table><tr><th colspan="5">Type Field</th><th rowspan="2">Descriptor Type</th><th rowspan="2">Description</th></tr><tr><th>Decimal</th><th>11</th><th>10 E</th><th>9 W</th><th>8 A</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>Data</td><td>Read-Only</td></tr><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>Data</td><td>Read-Only, accessed</td></tr><tr><td>2</td><td>0</td><td>0</td><td>1</td><td>0</td><td>Data</td><td>Read/Write</td></tr><tr><td>3</td><td>0</td><td>0</td><td>1</td><td>1</td><td>Data</td><td>Read/Write, accessed</td></tr><tr><td>4</td><td>0</td><td>1</td><td>0</td><td>0</td><td>Data</td><td>Read-Only, expand-down</td></tr><tr><td>5</td><td>0</td><td>1</td><td>0</td><td>1</td><td>Data</td><td>Read-Only, expand-down, accessed</td></tr><tr><td>6</td><td>0</td><td>1</td><td>1</td><td>0</td><td>Data</td><td>Read/Write, expand-down</td></tr><tr><td>7</td><td>0</td><td>1</td><td>1</td><td>1</td><td>Data</td><td>Read/Write, expand-down, accessed</td></tr><tr><td></td><td></td><td>C</td><td>R</td><td>A</td><td></td><td></td></tr><tr><td>8</td><td>1</td><td>0</td><td>0</td><td>0</td><td>Code</td><td>Execute-Only</td></tr><tr><td>9</td><td>1</td><td>0</td><td>0</td><td>1</td><td>Code</td><td>Execute-Only, accessed</td></tr><tr><td>10</td><td>1</td><td>0</td><td>1</td><td>0</td><td>Code</td><td>Execute/Read</td></tr><tr><td>11</td><td>1</td><td>0</td><td>1</td><td>1</td><td>Code</td><td>Execute/Read, accessed</td></tr><tr><td>12</td><td>1</td><td>1</td><td>0</td><td>0</td><td>Code</td><td>Execute-Only, conforming</td></tr><tr><td>13</td><td>1</td><td>1</td><td>0</td><td>1</td><td>Code</td><td>Execute-Only, conforming, accessed</td></tr><tr><td>14</td><td>1</td><td>1</td><td>1</td><td>0</td><td>Code</td><td>Execute/Read-Only, conforming</td></tr><tr><td>15</td><td>1</td><td>1</td><td>1</td><td>1</td><td>Code</td><td>Execute/Read-Only, conforming, accessed</td></tr></table>	Type Field					Descriptor Type	Description	Decimal	11	10 E	9 W	8 A	0	0	0	0	0	Data	Read-Only	1	0	0	0	1	Data	Read-Only, accessed	2	0	0	1	0	Data	Read/Write	3	0	0	1	1	Data	Read/Write, accessed	4	0	1	0	0	Data	Read-Only, expand-down	5	0	1	0	1	Data	Read-Only, expand-down, accessed	6	0	1	1	0	Data	Read/Write, expand-down	7	0	1	1	1	Data	Read/Write, expand-down, accessed			C	R	A			8	1	0	0	0	Code	Execute-Only	9	1	0	0	1	Code	Execute-Only, accessed	10	1	0	1	0	Code	Execute/Read	11	1	0	1	1	Code	Execute/Read, accessed	12	1	1	0	0	Code	Execute-Only, conforming	13	1	1	0	1	Code	Execute-Only, conforming, accessed	14	1	1	1	0	Code	Execute/Read-Only, conforming	15	1	1	1	1	Code	Execute/Read-Only, conforming, accessed
Type Field					Descriptor Type	Description																																																																																																																														
Decimal	11	10 E	9 W	8 A																																																																																																																																
0	0	0	0	0	Data	Read-Only																																																																																																																														
1	0	0	0	1	Data	Read-Only, accessed																																																																																																																														
2	0	0	1	0	Data	Read/Write																																																																																																																														
3	0	0	1	1	Data	Read/Write, accessed																																																																																																																														
4	0	1	0	0	Data	Read-Only, expand-down																																																																																																																														
5	0	1	0	1	Data	Read-Only, expand-down, accessed																																																																																																																														
6	0	1	1	0	Data	Read/Write, expand-down																																																																																																																														
7	0	1	1	1	Data	Read/Write, expand-down, accessed																																																																																																																														
		C	R	A																																																																																																																																
8	1	0	0	0	Code	Execute-Only																																																																																																																														
9	1	0	0	1	Code	Execute-Only, accessed																																																																																																																														
10	1	0	1	0	Code	Execute/Read																																																																																																																														
11	1	0	1	1	Code	Execute/Read, accessed																																																																																																																														
12	1	1	0	0	Code	Execute-Only, conforming																																																																																																																														
13	1	1	0	1	Code	Execute-Only, conforming, accessed																																																																																																																														
14	1	1	1	0	Code	Execute/Read-Only, conforming																																																																																																																														
15	1	1	1	1	Code	Execute/Read-Only, conforming, accessed																																																																																																																														

GDT 存在于内存之中，系统如何知道内存那一片是 GDT？答案是 GDTR 寄存器。GDTR 寄存器的作用是为系统指定 GDT 表的起始位置与界限，通过 lgdt 指令可以对 GDTR 寄存器赋值。GDTR 寄存器的结构如图 3 所示。当我们准备好了 GDT 表并且加载了 GDTR 寄存器后，一切已经准备完毕，我们只需拨动开关，就可以摆脱实模式，进入保护模式了。这个开关就在 CR0 寄存器中。CR0 寄存器的第 0 位（即 PE 位），就是我们所谓的“开关”，当它被置为 1 时就表明 CPU 当前是运行于保护模式下了。



图 3：GDTR 寄存器构成

前文中述及，段选择子就相当于某个段描述符在 GDT 表中的偏移量，但又比偏移量要稍微复杂些，其结构为：



其中：TI=0，表示从全局描述符表 GDT 中读取描述符；TI=1，指示从局部描述符表 LDT 中读取描述符（局部描述符表 LDT 与 GDT 结构相同，这里暂时不展开做过多解释）。RPL 表示实现特权级比较时需要用到的两位，关于特权级的问题暂时不作解释。例如：当 selector = 0x8 时，表示选择全局描述符表中的第 1 个描述符，特权等级为 0。

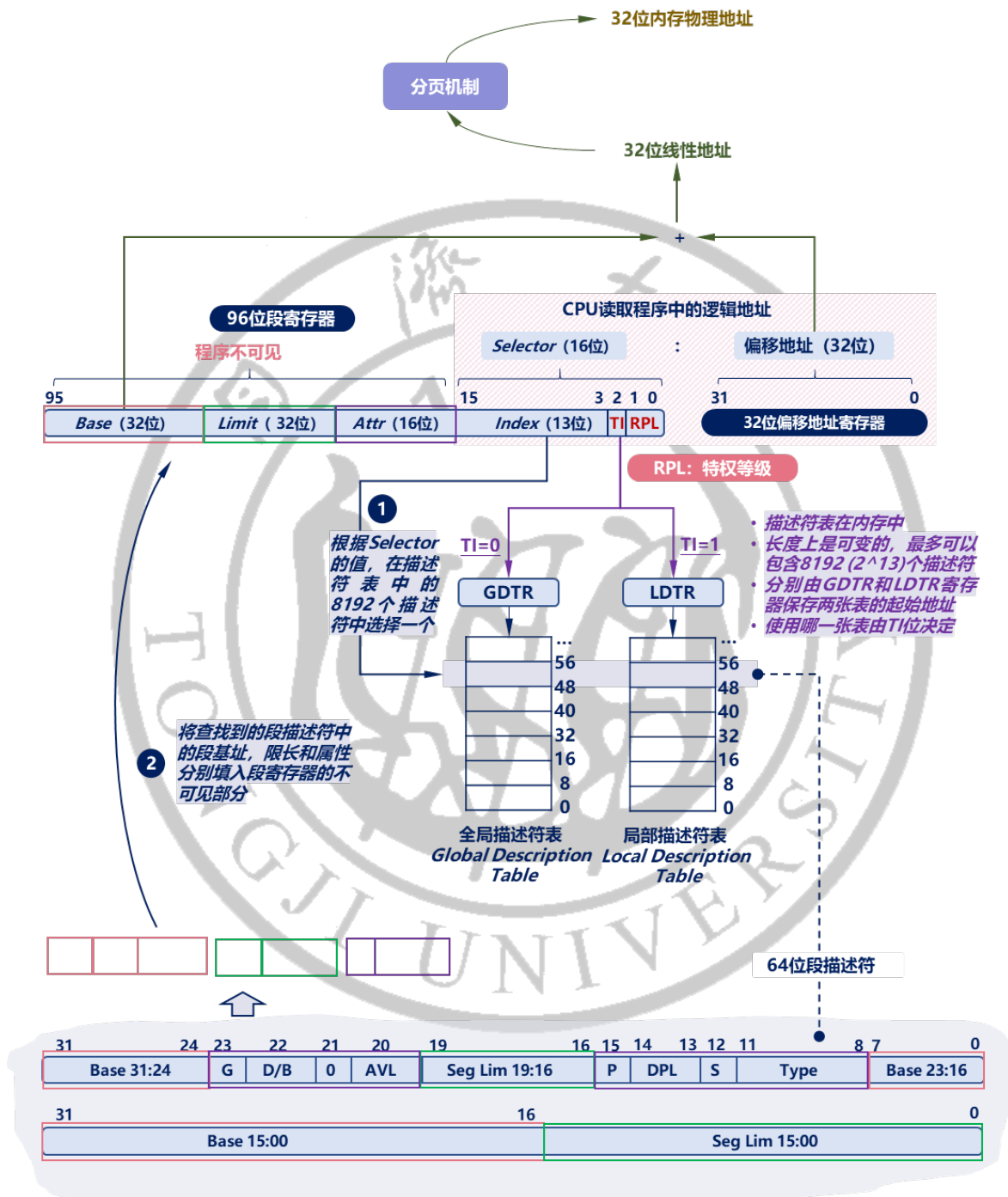


图 4：保护模式下线性地址（逻辑地址）的形成过程

有了段描述符之后，保护模式下，采用“段选择子:段内偏移地址”的形式表示的程序地址（逻辑地址）经由图 4 所示的过程转换为线性地址。图中，段选择子其实是系统中 96 位的段寄存器的后 16 位，也只有这 16 位是程序可见的。

仅仅到这一步，强大的保护模式的地址管理还没有结束，还有更伟大的分页机制。我们将在后续章节中继续探索。在系统没有开启分页模式的条件下，线性地址和物理地址是相等的，可以使用线性地址的值直接访问物理内存。

3.2. PC 的启动过程

PC 在系统上电之后首先执行的是 BIOS 程序。下面这条指令是 BIOS 执行的第一条指令：

```
[F000 : FFF0] : jmp    $0xF000, $0xE05B
```

通过这条指令我们可以得到如下的结论：

(1) 由上述指令的逻辑地址[F000:FFF0]可知，PC 开始执行时，CS = 0xF000，IP = 0xFFF0；根据图 1 所示的计算方法可知，CS = 0xF000，IP = 0xFFF0 对应的物理地址为：0xFFFF0，正好指向 1M 的物理内存中为 ROM BIOS 保留的 64K 字节的倒数 16 个字节。

(2) 在物理地址 0xFFFF0 处，仅仅存放一条 jmp 指令，毕竟，在 16 个字节内能完成多少工作？跳转的地址是：CS = 0xF000，IP = 0xE05B。该地址为 BIOS 自检程序的入口。上述过程如图 5 中的①所示。BIOS 自检程序是由硬件生产厂商预置在系统 ROM 中的一段引导程序，BIOS 自检程序会完成一系列对外设的检测，包括初始化 PCI 总线和 BIOS 知道的所有重要设备，建立一个中断描述符表和初始化各种设备，如 VGA 显示器。这就是你在 QEMU 窗口中看到的“Starting SeaBIOS”消息的来源。

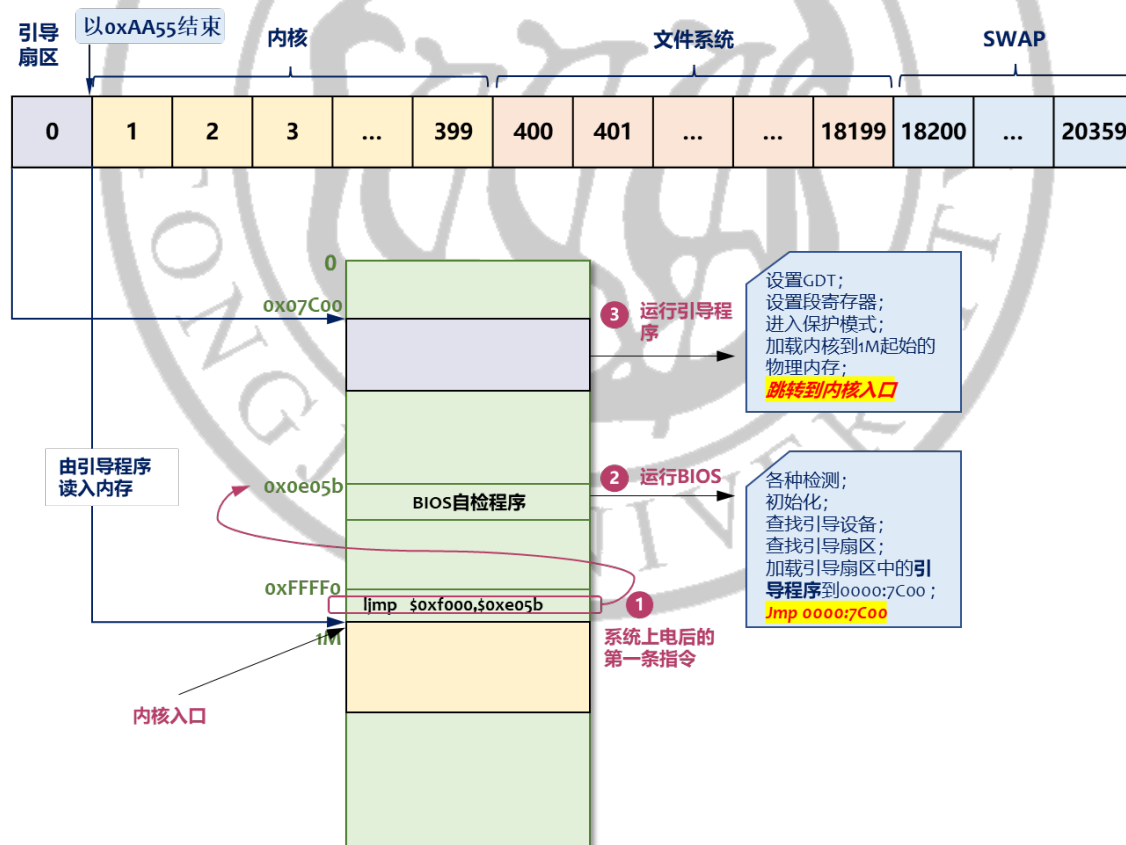


图 5：UNIX V6++的磁盘分布及启动过程

此后，BIOS 会搜索可引导设备，如软盘、硬盘或 CD-ROM。以硬盘为例。PC 的硬盘被划分为称为扇区的 512 字节区域。如果硬盘是可引导的，则第一个扇区称为引导扇区（Boot Sector），这是 BootLoader 代码所在的位置。当 BIOS 找到可引导的硬盘时，通过检查存储设备上的第一个扇区（0#柱面 0#磁头 0#扇区）中的末尾

2 个字节，也就是第 511 和 512 字节是否为 0x55 和 0xAA，来判断它是否为一个引导扇区。如果是，则 BIOS 自检程序会将该引导扇区的 512 字节加载到物理地址 0x7C00 到 0x7DFF 的内存中，并跳转到 CS:IP = 0000:7C00 处开始执行 Bootloader 的第一条指令，正式将控制权交给引导扇区中的 Bootloader 代码。这一过程如图 5 中的②所示。至此，BIOS 程序完成了所有工作，Bootloader 开始执行。

Bootloader 完成的工作包括设置段表，设置段寄存器，进入保护模式，即：开启了 1M 以上的物理地址空间。紧接着，从 1#扇区开始，将内核代码读入 1M 开始的物理内存中。然后跳转到 1M 处的内核入口。至此，机器的控制权完全传递给操作系统。这一过程如图 5 中的③所示。

以上就是 PC 中使用的英特尔 8088 处理器启动的方式。由于 PC 中的 BIOS“硬连线”到物理地址范围 0xF0000 ~ 0xFFFFF 的 64KB 地址空间，这种设计确保了 BIOS 在通电或系统重启后始终首先控制机器——这一点至关重要。因为通电时，机器的 RAM 中没有处理器可以执行的任何其他软件。

我们的实验平台使用的 QEMU 仿真器自带 BIOS，它将 BIOS 同样放置在模拟的物理地址空间 0xFFFF0 的这个位置。在处理器重置时，（模拟）处理器进入实模式，并将 CS 设置为 0xF000，将 IP 设置为 0xFFFF0，以便从该（CS:IP）地址开始执行整个内核的引导过程。

3.3. UNIX V6++的重新编译



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[vesper_center_1@archlinux unix-v6pp-tongji]$ make all

[bin/..] > [info] 切换路径。
[bin/../etc] > [info 5] 上传成功: v6pp_splash.bmp
[bin/../etc] > [info] 切换路径。
[bin/../etc/..] > [info 5] 上传成功: Shell.exe
[bin/../etc/..] > bye!
cp target/img-workspace/c.img target/
build success (unix-v6pp-tongji).
[vesper_center_1@archlinux unix-v6pp-tongji]$

```

图 6：重新编译 UNIX V6++的源代码

如图 6 所示，在 vscode 的 Terminal 窗口中直接输入 make all 命令并回车（注意当前所在路径），将完成 UNIX V6++代码的重新编译，并给出编译成功的提示。

如果多次重新编译后，发现 UNIX V6++的运行结果没有变化，可以尝试先输入“make clean”指令，以全部删除之前编译的结果，然后再输入“make all”重新编译。

3.4. 关于 X86 的寄存器及基本的汇编指令

通过自己查阅资料或者阅读教师下放的补充材料，已基本掌握 X86 体系结构下的核心寄存器构成及基本汇编语言程序的阅读和理解。

4. 实验内容

Bootloader 因其与系统启动引导过程紧密相关，通常只能采用汇编语言进行编写，于是也就成为了系统中一处使用较多汇编开发的模块。为了减少使用汇编进行开发的难度，我们应该使 Bootloader 所负责的工作最小化，完成将位于磁盘上的内核映像加载到内存中的特定位置，以及为完成该任务的稍许准备工作，

即为 Bootloader 的全部职责。然而，我们并不准备一蹴而就直接实现一个功能完备的最终版本 Bootloader，而是将采取循序渐进的方式，从最基本的让 Bootloader 显示字符开始，逐步加入进入保护模式，读取磁盘，加载内核到内存的能力，直至完整功能的 Bootloader。

找到“/src/boot/boot.asm”文件，先将其原有内容保存备份，然后全部删除，接下来你就一步步尝试完成你自己的 BootLoader 吧！

4.1. BOOT LOADER VERSION 0: 实模式下的“HELLO WORLD!”

这个版本 Bootloader 的目标是：在屏幕上打印“Hello world!”字样。下面的代码 2 实现这样的功能，它调用了 BIOS 0x10# 中断，在屏幕上显示字符串。

代码 2：“Hello World!” Bootloader

```
org 0x7c00          ; 对齐到 0x7c00, BIOS 从这个入口启动 OS

[BITS 16]           ; 实模式
start:
    mov ax,cs
    mov ds,ax        ; 置数据段地址
    mov es,ax
    mov bp,msg        ; es:bp = msg
    mov cx,13         ; 串长 = 13
    mov dx,0x1800     ; DH,DL = 起始行、列 = 24,0
    mov bx,0x000c     ; BH = 页号 = 0; BL = 样式 = 0xc = 黑底红字
    mov ax,0x1301     ; AH = 0x13, 调用 0x10 中断功能为显示字符串; AL = 0x1, 光标跟随移动
    int 0x10          ; 调用 BIOS 中断#10 打印文字
loop:
    jmp loop          ; 死循环
msg db 'Hello world!'

times 510-($-$$) DB 0
DW 0xAA55             ; 最后两行用于补齐 512 字节大小
```

代码 2 的运行结果如图 7 所示。关于代码 1 有以下两点说明：

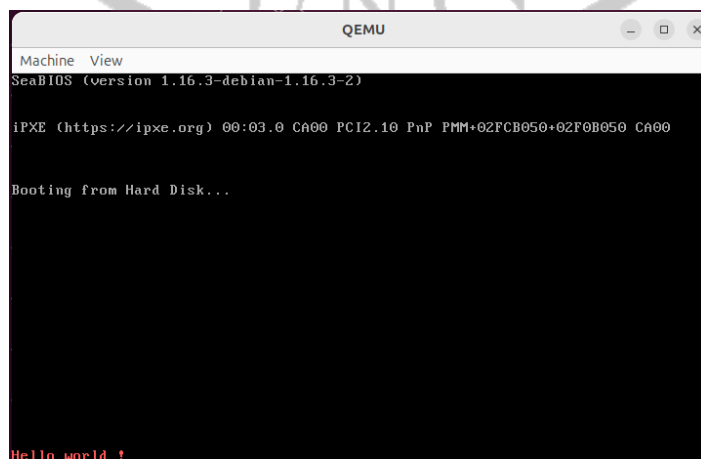


图 7：代码 2 运行结果

(1) `org` 指令的作用是指示程序将要被加载到内存的起始地址，并对程序段中每一次对内存的访问都加上偏移量 `0x7C00`（没有 `org` 指令时的偏移量为 0）。换句话说，取定 `org 0x7C00` 而非其它任意值是为了与 PC 启动过程中引导扇区加载到物理内存 `0x7C00` 处的规范相互吻合。举例来说，如果取值 `org 0x8c00`，而引导扇区却被加载到 `0x7C00`，这意味着每次对内存的引用加上 `0x8c00` 偏移量，显然访问不到正确的引导扇区中的数据。

(2) 代码中我们将引导扇区 512 字节中的最后 2 个字节填充为 `0x55AA`，这是判断一个扇区是否为引导扇区的标志字节，BIOS 自检程序通过判断这 2 个标志字节来决定是否加载引导扇区到内存中，也就是说，无论我们编写的 Bootloader 逻辑如何正确，如果缺少了这 2 个标志字节，那么它连被加载到内存中执行的机会都没有。语句：

`times 510-($-$$) DB 0`

是为了填充一系列的 0 直至塞满前 510 个字节，然后补上末尾的 `0x55AA` 两个标志字节。`$$` 表示当前节（section）起始处的字节偏移值，而 `$` 表示的是当前指令的字节偏移值，`$` 和 `$$` 的具体值要到编译时才可以确定。具体在我们代码中，当前节起始处的字节偏移值 `$$` 应该为 0，而 `$` 的值则为当前其所属的 `times` 语句编译后的字节偏移，那么两者相减（`$-$$`）的值便是整段程序编译后占据的字节长度，总共 510 字节减去 Bootloader 中程序段占据的长度即为需要填充的 0 的个数。这便是 `510-($-$$)` 取值的由来。

4.2. BOOT LOADER VERSION 1: 进入保护模式

先前 Version 0 中的 Bootloader 仅仅演示了系统启动时成功执行我们编写的 Bootloader 的效果，在屏幕上打印出一串字符，但是还毫不具备我们需要的功能。这样的 Bootloader 有意义吗？显然是有的，通过它我们至少知道如何编写一个能够被正确执行到的引导扇区，知道任何一个 Bootloader 所需具备的要素有哪些。在此基础上，我们需要将从标识符 `start` 开始的显示字符串的代码替换成有实际意义的功能。而本节 Version 1 版本的 Bootloader 的目标是：

(1) 从实模式进入保护模式，突破 1MB 空间的限制；

(2) 由于启用保护模式后，BIOS 调用不再可用，我们需要新的方法在屏幕上显示反馈，方便我们验证。

一般来说，从实模式进入保护模式需要以下四个关键步骤：

(1) 建立 GDT 表（全局描述符表）与 GDTR 数据结构

(2) 加载 GDT（即设置 GDTR 寄存器指向 GDT）

(3) 设置 CR0 寄存器中的 PE（Protection Enable）位来启用保护模式

(4) 一条 JMP 指令，由实模式跳入保护模式

下面，我们将详细解释以上几步如何实现。

首先，我们需要建立一个 GDT 表，其中包含 5 个 GDT 表项，即 5 个段描述符（Segment Descriptor），每个描述符所描述段的属性在表 4 中给出，和代码 3 中的 GDT 数据结构定义完全等价。从中可以看到，GDT 表中第 0 项必须全部设置为 0，以此构造一个空描述符作为占位符。GDT 中有效描述符从第一项开始，我们总共定义了 boot 运行时使用的代码段和数据段，内核初始化阶段的代码段和数据段四个段描述符。

boot 运行时使用的代码段（对应的段选择子为 `0x8`）和数据段（对应的段选择子为 `0x10`）的属性大致相同，起始地址都是 `0x00000000`，段长为 `0xFFFFF`，特权级 DPL 都为 0 级，区别在于：代码段是执行/可读，数据段是可读/写。

内核初始化阶段的代码段（对应的段选择子为 0x18）和数据段（对应的段选择子为 0x20）情况基本相同，唯一的区别在于起始地址都是 0x40000000，为什么设置这样的起始地址，我们将在 4.4 节中给出解释。

表 4：GDT 表中段描述符的设置

段描述符	起始地址	限长	G	D/B	o	AVL	P	DPL	S	Type
空描述符	0x00000000	0	0	0	0	0	0	00	0	0
boot 运行时使用的代码段	0x00000000	0xffff	1	1	0	0	1	00	1	A-执行/可读
boot 运行时使用的数据段	0x00000000	0xffff	1	1	0	0	1	00	1	2-可读/写
内核初始化阶段的代码段	0x40000000	0xffff	1	1	0	0	1	00	1	A-执行/可读
内核初始化阶段的数据段	0x40000000	0xffff	1	1	0	0	1	00	1	2-可读/写

代码 3：建立 GDT 表

```

gdt:                ; gdt 表头的占位描述符
    dw 0x0000
    dw 0x0000
    dw 0x0000
    dw 0x0000        ;GDT 表第 0 项必须为全零

; boot 运行时使用的代码段，段选择子 0x8
    dw 0xFFFF        ; limit(L15~0)=0xffff
    dw 0x0000        ; base(B15~0)=0x0000
    dw 0x9A00        ; P=1, 段存在内存; DPL=0, 特权级; TYPE=1010, 执行/可读; B23~16 = 0x00
    dw 0x00CF        ; B31~24 = 0x00; ; G=1, 段限单位为 4KB; D/B=1, 32bit; L19~16=0xf

; boot 运行时使用的数据段，段选择子 0x10
    dw 0xFFFF        ; limit(L15~0)=0xffff
    dw 0x0000        ; base(B15~0)=0x0000
    dw 0x9200        ; P=1, 段存在内存; DPL=0, 特权级; TYPE=0010, 可读/写; B23~16 = 0x00
    dw 0x00CF        ; B31~24 = 0x00; G=1, 段限单位为 4KB; D/B=1, 32bit; L19~16=0xf

; 内核初始化阶段的代码段，段选择子 0x18
    dw 0xFFFF        ; limit(L15~0)=0xffff;
    dw 0x0000        ; base(B15~0)=0x0000
    dw 0x9A00        ; P=1, 段存在内存; DPL=3, 特权级; TYPE=0010, 可读/写; B23~16 = 0x00
    dw 0x40CF        ; B31~24 = 0x40; G=1, 段限单位为 4KB; D/B=1, 32bit; L19~16=0xf

; 内核初始化阶段的数据段，段选择子 0x20
    dw 0xFFFF        ; limit(L15~0)=0xffff
    dw 0x0000        ; base(B15~0)=0x0000
    dw 0x9200        ; P=1, 段存在内存; DPL=3, 特权级; TYPE=0010, 可读/写; B23~16 = 0x00
    dw 0x40CF        ; B31~24 = 0x40; G=1, 段限单位为 4KB; D/B=1, 32bit; L19~16=0xf

; 此处多增加一个显存段，用于屏幕输出测试

```

```

dw 0xFFFF      ; limit(L15~0)=0xffff;
dw 0x8000       ; base(B15~0)=0x8000
dw 0x920B       ; P=1, 段存在内存; DPL=0, 特权级; TYPE=0010, 可读/写; B23~16 = 0x0B
dw 0x00CF       ; B31~24 = 0x00; G=1, 段限单位为 4KB; D/B=1, 32bit; L19~16=0xf

```

; GDT End

这里需要说明的是，代码 3 的最后还多定义了一个显存段（对应的段选择子为 0x28），起始地址为 0xB8000（见表 1 关于 PC 的 1M 物理内存分布），段长为 0xFFFFF，其作用适用于保护模式下向显存写入信息，这些信息会反馈在屏幕上，以此验证程序的正确性。该段可在这部分实验后删除。

随后，我们需要建立一个 GDTR，它包含两部分信息，GDT 表的起始地址和限长，简略的说，它解决了系统应该到哪里找到 GDT 的问题。GDTR 数据结构长度为 6 个字节，其中 2 字节的限长（Limit）在前，4 字节的起始地址（Base Address）在后。建立 GDTR 代码片段（代码 4 所示）需要紧随建立 GDT 的代码片段，这样可以通过 \$-gdt 计算 GDT 表总长度。

代码 4：建立 GDTR 数据结构

```

gdt:      ;gdt 数据结构
          dw $-gdt      ; GDT Limit
          dd gdt         ; GDT Base Address

```

在准备完 GDT 表和 GDTR 这两个保护模式所必需的数据结构之后，意味着进入保护模式前的一切准备工作就绪了。接下来就可以执行进入保护模式的操作了，具体方法如代码 5 所示。

代码 5：进入保护模式

```

lgdt [gdt]      ;根据 gdt 所指示的内容加载 GDTR 寄存器
cli             ;关中断

```

```

;打开 a20 地址线
in al,92h
or al,00000010b
out 92h, al

```

;设置 CR0 寄存器中的 PE（Protection Enable）位来启用保护模式

```

mov eax, cr0
or eax, 1
mov cr0, eax

```

jmp dword 0x8: _startup;

;section .code32

[BITS 32]

_startup:

```

    mov ax, 0x10
    mov ds, ax
    mov es, ax
    mov ss, ax

```

这里必须要说明以下流程：

(1) 使用 `lgdt` 加载 `gdt` 数据结构到 `GDTR` 寄存器，系统通过 `GDTR` 了解到 `GDT` 到底在哪里。

(2) 在进入保护模式之前必须要屏蔽中断（注意跳入保护模式前有 `cli` 关中断指令），这样做是因为保护模式下还没有准备好响应外设中断的处理函数，假如此时有中断进入系统将会崩溃，只有将各种中断对应的处理函数设置完成后才可以开中断。读者可以尝试将 `cli` 指令去除，观察程序运行的结果。

(3) 打开 `A20` 地址线，开启 `1M` 以上的物理内存。这样做的原因感兴趣的读者可自行查阅资料，此处不再展开。

(4) 通过修改 `CR0` 控制寄存器的 `PE` 比特位（相当于保护模式开关）来启用保护模式。

(5) 一条 `JMP` 指令，跳转入保护模式。通过 `jmp CS:EIP` 的形式，修改代码段寄存器 `CS = 0x8`（`15~3` 位为 `1`，`2~0` 位为 `0`，意为使用表 4 的第 1 项段描述符：`boot` 运行时使用的代码段，特权等级为 `0`），`EIP = _start`，该 `JMP` 指令使程序跳转至 `_start` 标号处在保护模式下继续执行。

(4) 此刻，其他的段寄存器（`DS`、`ES`、`SS`）值并不是段选择子，而是从先前实模式下继承来的段基地址，我们需要修正它们：`_start` 后面几句，修改数据段寄存器（`DS`、`ES`、`SS`）= `0x10`。（`15~3` 位为 `2`，`2~0` 位为 `0`，意为使用表 4 中的第 2 项段描述符：`boot` 运行时使用的数据段，特权等级为 `0`）。

最后，我们需要看到效果，就需要往屏幕上输出些信息。而这里需要新办法显示回馈信息。为什么不继续采用 `Version 0` 中的方式呢？这是因为原先实模式下 `BIOS` 中断在保护模式下不再可用，我们需要另外的方式：直接往显存中写字符，显存地址开始于 `0xB8000`，直接向其中写值便可显示在屏幕上。最后一条 `hlt` 指令的目的是让 `BootLoader` 暂时停一下，因为我们还有后续很多工作没有完成。

代码 6：显示回馈字符“P”

```
mov ax, 0x28
mov gs, ax
mov ah, 0xc; 黑底红字
mov al, 'P'
mov [gs:80*2*24], ax
; 已经在 GDT 中添加了第三个段，段基址为 0xB8000（显存起始地址）
; 主要为了测试 GDT 起了作用
; 上面语句在屏幕第 25 行输出红色的 P
; （80 为每行字符数，*2 因为偏移是字节量）
hlt
```

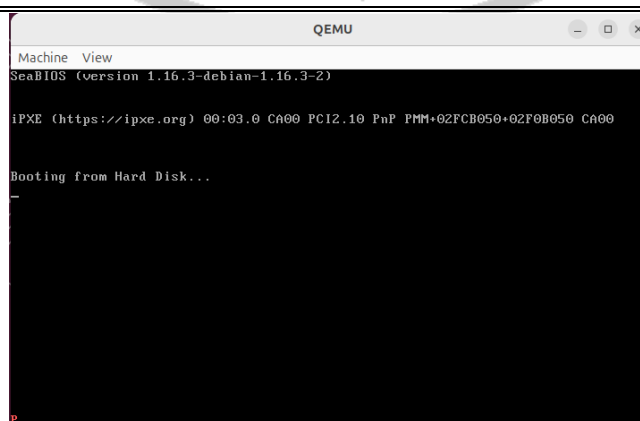


图 8：进入保护模式的 `BootLoader` 运行结果

请读者读懂并理解代码 3 ~ 代码 6，综合这三段代码，修改代码 2。修改过程中，请务必注意三段代码的前后位置，代码 2 中哪些部分需要被覆盖，哪些部分需要保留。再次编译运行后，可以获得如图 8 所示的运行结果。屏幕左下方出现一个红色的大写字母“P”。

这部分代码在运行和调试过程中需要注意以下几个问题：

- (1) 图 8 中的运行结果必须稳定，若不停闪屏，则说明代码有问题，需要查找并改正错误；
- (2) 由于 BIOS 调试难度较大，不熟悉的同学可以尝试在代码中的不同位置插入 hlt 停机指令，以确定程序无法正确停止而不停屏闪的确切位置；
- (3) 如前所述，必要时候，可以尝试先输入“make clean”指令，以全部删除之前编译的结果，然后再输入“make all”重新编译。

4.3. 加载内核映像

前述 Bootloader 版本，无论是显示一行欢迎字符或者是从实模式进入保护模式，都并非 Bootloader 的最终目的，而仅仅是一些准备工作。Bootloader 最终目标则是将内核从磁盘加载到内存中。此目标看似简单，其实包含着几个有待解决的问题。内核存放在磁盘上什么地方？内核本身是什么格式？大小是多少？以及如何进行磁盘的读取？只有在这些问题都得到回答之后，Bootloader 才可能正确无误地在磁盘上找到内核并加载进内存。为此我们需要进行一些明确的设定。

UNIX V6++ 内核在磁盘中的位置如图 5 所示。磁盘扇区编号从 0# 开始，引导扇区占据 0# 扇区；内核紧随其后占据从 1# 扇区开始的连续 399 个扇区，假定内核占据的扇区不越过第 399# 扇区（就 V6++ 而言预留的空间已经足够）；从 400# 扇区开始为文件系统。

内核采用纯二进制格式，而不是 PE 或 ELF 等可行性格式，从而避免了让 Bootloader 从 PE 或 ELF 头中分析出程序入口地址和段信息的麻烦。需要简单解释一下的是，内核由许多个源代码文件经过编译、链接最终获得一个可执行格式（PE 或 ELF）文件，然后将 PE 或 ELF 头、符号表、调试信息等从可执行文件中去除，仅提取并保留内核的代码段、数据段、bss 段等用于执行的必要数据，这才得到我们需要的纯二进制格式的内核。

对 V6++ 的 Bootloader 具体而言，就是将内核从磁盘加载到 1M 起始的物理内存中。此部分代码由于涉及较多的磁盘控制器操作，读者可不必关注过多细节，了解基本的操作流程即可。

4.3.1. _LOAD_KERNEL

立足于之前版本已经进入保护模式的基础上，我们将通过代码 7 来完成从磁盘加载内核到 1M 物理内存处的目标。

代码 7：_load_kernel

```
[BITS 32]
mov ecx, KERNEL_SIZE ;cx = 扇区数 KERNEL_SIZE, 作为 loop 的次数
mov eax, 1             ;LBA 寻址模式下 sector 编号从 0 开始。#0 是引导扇区，#1 扇区开始才
                        ;是 kernel 的首扇区
mov ebx, 0x100000      ;目标存放地址从 1M 处开始，每次 loop 递增 512 bytes

_load_kernel:
    push eax
```

```

inc eax

push ebx
add ebx, 512
call _load_sector
loop _load_kernel

```

```

;section .data

```

```

KERNEL_SIZE      equ      (398)

```

由给出的代码不难发现，`_load_kernel` 函数完成了整个内核加载（暂且称之为函数，确切说应该只是一个标号）。之前已经明确设定内核位置和大小，我们编写 `_load_kernel` 使之从 1#扇区开始读取磁盘，直到第 399#扇区结束。因此我们定义常量 `KERNEL_SIZE = 398`，它决定了循环执行 `_load_kernel` 的次数，每一次循环调用 `_load_sector` 函数读取一个扇区的数据。`_load_sector` 函数进行真正的磁盘扇区的读取，而 `_load_kernel` 并不涉及磁盘 I/O 操作，后面会单独对 `_load_sector` 函数进行分析。

在 `_load_kernel` 每一次循环调用 `_load_sector` 时，需要为 `_load_sector` 准备两个参数：当前读取扇区号和内核加载的目标内存地址。`eax` 寄存器中存放当前要读取扇区号，从 1#扇区开始，每次 `loop` 递增 1；内核加载目标地址存于 `ebx` 寄存器中，初始从 1M 内存处开始，每次 `loop` 时加载目标地址递增 512 字节，使得下一磁盘扇区的数据读入到后继 512 字节内存中。`_load_kernel` 函数循环执行 `KERNEL_SIZE` 次，完成加载内核至 1M 内存的工作。

整个 `_load_kernel` 函数的过程如果转换为等价的伪代码，基本上会是下面这样：

代码 8: `_load_kernel` 的伪代码

```

void _load_kernel()
{
    int sector = 1;
    int address = 0x100000;           //加载到 1M 目标内存地址
    for(int i = 0; i < KERNEL_SIZE; i++)
    {
        _load_sector(address, sector); //调用 _load_sector 读取一个扇区数据
        address += 512;
        sector += 1;
    }
}

```

定义常量 `KERNEL_SIZE` 为内核可能占用的最大扇区数目是为了方便，该数值只要大于内核实际占用扇区数都可正确运行。刚开始的时候，内核规模可能很小，只有二、三十个扇区不到，`KERNEL_SIZE` 取值过大会读入多余的空扇区，不会导致任何副作用，但是这至少避免了我们每次内核大小增长时都要去修改读取磁盘扇区的数目。

4.3.2. `_LOAD_SECTOR`

前文中提及，`_load_kernel` 函数通过多次调用 `_load_sector` 函数，一次一个扇区的将内核加载到 1M 开始的物理内存中。本节我们就来具体看一下 `_load_sector` 函数的实现细节。代码 9 给出了其完整内容。

代码 9: _load_sector 的代码

_load_sector:

push ebp
mov ebp, esp

push edx
push ecx
push edi
push eax

mov al,1 ;读 1 个扇区
mov dx,1f2h ;向扇区数寄存器 0x1f2 中写入 1
out dx,al

mov eax,[ebp+12] ;[ebp+12]对应代码 6 中 mov eax,1 push eax 指令入栈的值
;eax 中为要读入的扇区号
;LBA28 (Linear Block Addressing) 模式输入扇区号的 Bits 7~0,
;共 28 Bits 扇区号

inc dx ;扇区号寄存器 0x1f3

out dx,al
shr eax,8 ;LBA28(Linear Block Addressing)模式输入扇区号的 Bits 15~8 放入 AL 中,
;共 28 Bits 扇区号

inc dx ;Port: DX = 0x1f3+1 = 0x1f4

out dx,al
shr eax,8 ;LBA28(Linear Block Addressing)模式输入扇区号的 Bits 23~16 放入 AL 中,
;共 28 Bits 扇区号

inc dx ;Port: DX = 0x1f4+1 = 0x1f5

out dx,al
shr eax,8
and al,0x0f
or al,11100000b

;Bit(7 和 5)为 1 表示是 IDE 接口,
;Bit(6)为 1 表示开启 LBA28 模式, Bit(4)为 1 表示主盘。
;Bit(3~0)为 LBA28 中的 Bit27~24 位

inc dx ;Port: DX = 0x1f5+1 = 0x1f6
out dx,al

mov al,0x20 ;0x20 表示读 1 个 sector, 0x30 表示写 1 个 sector

inc dx ;Port: DX = 0x1f6+1 = 0x1f7
out dx,al

.test:

in al,dx
test al,10000000b ;不断查询 0x1f7 寄存器, 看数据是否就绪
jnz .test

```

test al,00001000b ;检查数据是否出错
jz .load_error

mov ecx,512/4
mov dx,0x1fo
mov edi,[ebp+8] ;取得 call 前入栈参数[ebp+8] = 0x100000 = 1MB
rep insd
xor ax,ax
jmp .load_exit

.load_error:
mov dx,0x1f1
in al,dx
xor ah,ah

.load_exit:
pop eax ;对应上面的 PUSH EAX, 这里也加一句
pop edi
pop ecx
pop edx
leave ;Destory stack frame 等价于 2 条指令: MOV EBP, ESP 和 POP EBP
retn 8 ;retn 恒等于 ret, 后面加立即数 8, 是使 ESP = ESP + 8, 单位为 byte
;销毁调用_load_sector 前 2 次 PUSH 压入栈的参数:
;读取扇区号& 目的物理地址

```

`_load_sector` 函数完成从磁盘读取单个扇区到目标内存地址的功能。首先，我们需要明确的是 CPU，磁盘控制器和驱动器三者之间的工作方式。一般来说，对磁盘控制器编程来实现数据在磁盘与内存之间传送，主要有以下三种方式：

(1) Polling（轮询）：CPU 向磁盘控制器发送读/写命令，启动一次 I/O 操作。由于磁盘这类物理设备读写操作需要一定时间，而 CPU 并不能知道本次操作需要多少时间，于是在本次 I/O 操作完成之前唯一可以做的就是周期性地询问磁盘控制器“操作是否已经完成”；对于读操作，也就是是否已经完成读取扇区操作，数据已在缓冲区中。

(2) Interrupt-driven I/O（中断方式 I/O）：CPU 向磁盘控制器发送读/写命令启动一次 I/O 操作，然后 CPU 可以继续执行后续指令，而并不需要轮询磁盘控制器检查操作是否完成。等到 I/O 操作完成之后，磁盘控制器会向 CPU 发出中断告知本次 I/O 操作完成，数据已经在缓冲区中。

(3) DMA（Direct Memory Access，直接内存访问）：与中断方式 I/O 类似，由 CPU 向磁盘控制器和 DMA 控制器发送命令启动一次 I/O 操作，较中断方式 I/O 更先进的地方在于，整个数据传送不需要 CPU 介入，基本不占用 CPU 周期，而是由 DMA 控制器操纵直接在磁盘与内存之间进行成块数据传送。CPU 在此过程中可以执行后续指令，直至本次 I/O 操作完成后会向 CPU 发送中断信号，此时磁盘数据已经在内存中。

由上面可以看出, 中断方式 I/O 和 DMA 需要在中断打开的条件下使用, 这两种方式都会用发送中断信号方式通知 CPU, I/O 操作结束。那么, 问题来了, 前文中述及, 在进入保护模式之前必须要通过 cli 指令屏蔽中断, 而在_load_kernel 整个执行过程中始终是关中断的。因此_load_sector 函数要实现的是: 不借助 BIOS 中断以及关中断条件下读取一个扇区的数据。_load_sector 函数究竟是如何实现这一目标的呢? 答案只能是采取 Polling 方式进行读取工作。

另外需特别注意, 除了 DMA 方式中 CPU 收到中断表示数据已经读入到内存中(对读操作而言), Polling 以及 Interrupt-driven I/O 中读取完成并不表示数据已经在内存中, 仅仅意味数据已经从扇区中读入到磁盘控制器缓冲区中, 我们还需要将它从缓冲区读入内存, 这一环节在_load_sector 函数中有体现。

对磁盘控制器进行编程时, 需要发送参数和命令两部分信息。参数部分包括: 要读/写的起始扇区号、扇区个数、操作对象是主盘还是从盘; 命令部分需要指定当前 I/O 操作方式是读或写。具体的流程如下 1:

(1) 设置要读取的扇区数为 1, 每次读入一个扇区;

(2) 使用 LBA28 寻址方式, 分别将扇区号的 Bits 7~0 写入端口为 0x1f3 的寄存器, 扇区号的 Bits 15~8 写入端口为 0x1f4 的寄存器, 扇区号的 Bits 23~16 写入端口为 0x1f5 的寄存器, 扇区号的 Bits 27~24 写入端口为 0x1f6 寄存器的低 4 比特;

(3) 设置驱动器/磁头寄存器 (0x1f6) 的高 4 比特, 7 和 5 位为 1 表示是 IDE 接口, 6 位为 1 表示开启 LBA28 模式, 4 位为 1 表示访问主盘;

(4) 向命令寄存器输出 0x20 表示启动一次读操作;

(5) 由于采用 Polling 方式检查 I/O 操作是否完成, CPU 在此期间周期性地检查主状态寄存器 (0x1f7), 看数据是否就绪; 如果未就绪则忙等待直至就绪;

(6) 检查是否出错, 如果磁盘数据读取过程发生错误, 则立即退出;

(7) 若数据已经就绪在控制器缓冲区中, 则设置好目标内存地址, 使用 rep insd 指令将控制器缓冲区中的数据经由数据寄存器 (0x1f0) 读入内存。

上述通过控制器编程每次读取 1 个扇区大小的数据便是_load_sector 函数的执行过程, 其接受目标内存地址和目标读取扇区号两个参数, 能够在关中断情况下读取扇区数据, 配合_load_kernel 函数就实现了加载磁盘中占据连续多个扇区的内核映像到内存中的任务。

4.4. 为内核启动做准备

_load_kernel 循环执行完后, 内核映像全部被装载入内存, 此时, 为了给跳转至操作系统内核入口做准备, 需要完成重要的地址映射的工作。原因是: BootLoader 进入保护模式后, 使用的都是分段模式下的逻辑地址, 即: 使用“段选择子: 段内偏移地址”来表示程序的逻辑地址。此时, 由于使用了表 2 中 boot 运行时的代码段和数据段描述符, 实际得到的段基址为 0。所以, 线性地址在数值上是和逻辑地址中的段内偏移地址 (即: 寄存器中的地址) 是相同的。同时, 由于此时还没有开启分页模式, 线性地址就是物理地址, 即: CPU 直接使用线性地址去访问内存。因此, 实际造成的结果是 CPU 使用了和寄存器中的段内偏移地址相同的值去访问物理内存。

然而, 需要注意的是: UNIX V6++ 内核的执行起点为 greatstart, 在 sector2.asm 中声明 (见实验准备部分要求读者对 sector2.asm 的修改), 且被编译器放在了 3G+1M (即: 0xc0100000) 这一地址, 这一点可

¹ 关于磁盘控制器的硬件规范, 感兴趣的读者可自行查询资料了解详细内容。

通过链接脚本 `Kernel.Link.ld` 中的一部分内容（如代码 10 所示）看出。该部分代码完成的功能是：第一，将程序的起始对齐在 `0xc0100000`；其次，将 `sector2.o` 的正文段置于所有其他 `.o` 文件的正文段之前，保证了 `greatstart` 的起始位置是在 `0xc0100000`。

代码 10: `Kernel.Link.ld` 节选

```
ENTRY(greatstart)
SECTIONS
{
    .text 0xc0100000:
    {
        boot/sector2.o(.text)
        *(.text)
        *(.rodata*)
        . = ALIGN(4096);
    }
}
```

这样造成的后果是，一旦程序跳转到 `greatstart`，如果 CPU 还是直接使用 `3G+1M` 去访问物理内存，是不可能找到现在实际被装载到 `1M` 这一物理地址的内核入口的。因此，必须对这一地址进行修正。修正的方法包括（见代码 11）：

（1）通过向 `ds`, `es`, `ss` 送入段选择子 `0x20`（15~3 位为 4，2~0 位为 0，意为使用表 2 中的第 4 项段描述符：内核初始化阶段使用的数据段，特权等级为 0），完成各个段寄存器的设置。设置完成后，实际达到的效果是：段内偏移地址都将被加上一个 `0x40000000` 段基址。例如：`3G+1M` 的地址值计算得到的物理地址正好是 `1M`。

（2）通过一条 `jmp 0x18:0xc0100000` 完成 `cs` 寄存器的设置（15~3 位为 3，2~0 位为 0，意为使用表 2 中的第 3 项段描述符：内核初始化阶段使用的代码段，特权等级为 0）的同时，实现程序跳转到逻辑地址：`0x40000000+0xc0100000`。

（3）修正 `esp`。代码 11 中可以看到这样一条语句：

```
or esp, 0xc0000000
```

作用是为了修正 `esp` 寄存器的值，请读者自行考虑这样做的目的是什么。

请读者读懂并理解代码 7~ 代码 11，综合这几段代码，继续修改代码 2。同样的，修改过程中，请务必注意几段代码的前后位置，以及代码 2 中哪些部分需要被覆盖，哪些部分需要保留。再次编译运行后，如果出现如图 9 所示的界面，恭喜你，你已经完成了自己的内核启动模块。虽然看上去比原版的稍显简陋，但是不影响它功能齐全。

代码 11: 跳转到内核入口

```
;修改所有段寄存器
mov ax, 0x20
mov ds, ax
mov es, ax
mov ss, ax
or esp, 0xc0000000
```

```
jmp 0x18:0xc0100000 ;程序跳转
```

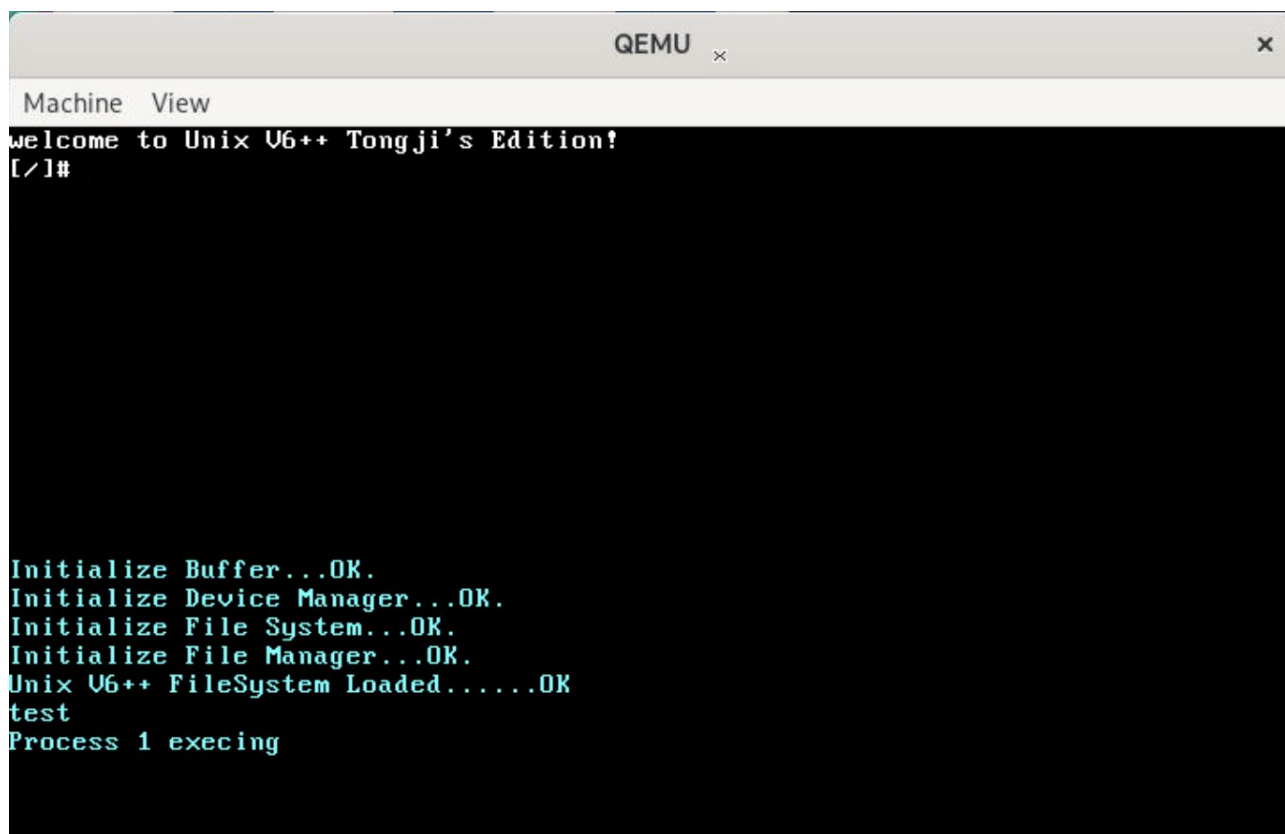


图 9: UNIX V6++启动界面

5. 实验报告要求

- (1) (1分) 完成实验 4.1, 截图说明结果;
- (2) (1分) 完成实验 4.2, 截图说明结果;
- (3) (1分) 完成实验 4.3, 截图说明结果;
- (4) (1分) 尝试回到实验 4.4 中的问题, 解释为什么最后需要修正寄存器 `esp` 的值。